ICARUS PoliTO - APA

HRE Design: Theory and models

Version 1-00

Politecnico
di Torino

09 December 2025

Author

Cristian Casalanguida

Mentions

Thanks to all DART and APA team members, this work is the conlcusion of three years of research and developement.
Thanks to Prof. Filippo Masseni for all the precious advices.

Abstract

To develop a Hybrid Rocket Engine it is important to possess the ability of predicting performances and costs in terms of propellant mass and volumes. To accomplish this task, different models for each component are needed.

Index

1. Introduction

The main focus will be on the description of the main components needed for the engine and how they are modeled. The components are the following:

- Oxidizer tank
- Feeding line
- Injector
- Combustion chamber
- Fuel grain
- Nozzle

Moreover, to find the best configurations, an optimization is required.

2. Tank

A self-pressurizing fluid is characterized by a high vapor pressure and evaporating it can reduce the pressure drop in blow-down tanks A tank full of oxidizer should be modeled according to the fluid properties; in particular, there are three main possible solutions:

- Self-pressurizing liquid
- Pressurized gas
- Constant pressure liquid with pressurant

2.1 Self-pressurizing

A self-pressurizing model for tank blow-down is taken from [5] and uses the CoolProp library in Python [1].
The model considers the tank to be isoentropic, considering the loss due to outflow:

$$s_{i+1} = \frac{s_i m_i - s_{L,i}\dot{m}_L dt - s_{G,i}\dot{m}_G dt}{m_i - \dot{m}_L dt - \dot{m}_G dt} \tag{2.1}$$

Knowing the mass and volume in every instant, it is possible to determine the total density. From density and total specific entropy we find vapor quality, pressure and temperature through the built-in fluid tables.

2.1.1 Functions

The functions to perform the calculations are in *Tank//tank_simulation.py*.

```
do_one_step(mdotL, mdotV, sL, sV, S, m, Q, oxidizer, Vtank, dt)
-> return m_new, mL_new, mV_new, Q_new, sL_new, sV_new, S_new, ptank_new, Ttank_new
```

Performs a single step of the blow-down. Input and output description in Table ??.

```
create_tank(m, Q, T, oxidizer, p=1e5)
-> return Vtank
```

Creates a self-pressurizing fluid tank: calculates each phase density and then combines them with vapor quality and total mass to get the volume. Input and output description in Table ??.

```
starting_conditions(m0, T0, Vtank, oxidizer)
-> ptank0, sL0, sV0, mL0, mV0, Q0, s0, S0
```

Calculates the inputs needed for the step function at time 0. Input and output description in Table ??.

Input at time *i*	**Output at time *i+1***
Liquid mass flow [kg/s]	Total mass [kg]
Vapor mass flow (through venting) [kg/s]	Liquid mass [kg]
Liquid specific entropy [J/(kgK)]	Vapor mass [kg]
Vapor specific entropy [J/(kgK)]	Vapor quality ($\frac{m_V}{m_L}$)
Total entropy [J/K]	Liquid specific entropy [J/(kgK)]
Total mass [kg]	Vapor specific entropy [J/(kgK)]
Vapor quality ($\frac{m_V}{m_L}$)	Total entropy [J/K]
Oxidizer dictionary {"OxidizerCP": CoolProp fluid name}	Tank pressure [Pa]
Tank volume [m^3]	Tank temperature [K]
Time step [s]	

Table 2.1: Step function input and output for self-pressurizing tank.

Input at time 0	**Output at time 0**
Total mass [kg]	Tank volume [m^3]
Vapor quality	
Ambient temperature [K]	
Oxidizer dictionary {"OxidizerCP": CoolProp fluid name}	

Table 2.2: create_tank input and output for self-pressurizing tank.

Input	**Output**
Total mass [kg]	Tank pressure [Pa]
Ambient temperature [K]	Liquid specific entropy [J/(kgK)]
Tank volume [m^3]	Vapor specific entropy [J/(kgK)]
Oxidizer dictionary {"OxidizerCP": CoolProp fluid name}	Liquid mass [kg]
	Vapor mass [kg]
	Vapor quality
	Total specific entropy [J/(kgK)]
	Total entropy [J/K]

Table 2.3: Starting conditions input and output for self-pressurizing tank.

2.2 Pressurized gas

A pressurized gas in blow-down has a pressure drop without any positive effect, the strategy is to have the highest starting pressure.

2.2.1 Functions

The functions to perform the calculations are in *Tank//tank_ simulation.py*.
They can be used without significant changes: to create the tank it is enough to insert 1 as quality and give the optional input of tank pressure.

2.3 Constant pressure

This configuration is composed of a pressure vessel, used to keep the pressure as constant as possible, and a propellant tank, pressurized at the regulation pressure to be kept. The model is based on the self-pressurizing one, considering the system to be isoentropic. In this case the entropy is transferred from the pressure vessel to the tank ullage.

2.3.1 Functions

The functions to perform the calculations are in *Tank//tank_pressurant_simulation.py*.

```
do_one_step(mdotL, mdotG, mdotpress, sL, sG, spress, mL, mG, mpress, T_tank,
propellant, pressurant, Vtank, Vpress, dt)
-> return mL_new, mG_new, mpress_new, sL_new, sG_new,
spress_new, ptank_new, Ttank_new, p_press_new, T_press_new
```

Calculates the needed output after one time-step. Pressure is calculated using entropy and density to confirm the constant pressure behaviour. Input and output description in Table ??.

Input at time *i*	**Output at time *i+1***
Liquid mass flow [kg/s]	Liquid mass [kg]
Gass mass flow (through venting and refill) [kg/s]	Gas mass [kg]
Pressurant mass flow (through refill) [kg/s]	Pressurant mass [kg]
Liquid specific entropy [J/(kgK)]	Liquid specific entropy [J/(kgK)]
Gas specific entropy [J/(kgK)]	Gas specific entropy [J/(kgK)]
Pressurant specific entropy [J/(kgK)]	Pressurant specific entropy [J/(kgK)]
Liquid mass [kg]	Tank pressure [Pa]
Gas mass [kg]	Tank temperature [K]
Pressurant mass [kg]	Pressure vessel pressure [Pa]
Tank temperature [K]	Pressure vessel temperature [K]
Propellant CoolProp name	
Pressurant CoolProp name	
Tank volume [m^3]	
Pressure vessel volume [m^3]	
Time step [s]	

Table 2.4: Step function input and output for constant pressure tank.

```
create_pressurant_tank(T0, P0, V_propellant, p_reg, pressurant)
-> return V0
```

This function creates the pressure vessel considering adiabatic discharge, input and output description in Table ??: from *First thermodynamic principle* we have

$$\Delta U = U_2 - U_1 = L = -pV_p \tag{2.2}$$

with

$$U_1 = m_0 c_v T_0$$

(values refer to pressure vessel' starting conditions)

$$U_2 = m_g c_v T_g + m c_v T$$

(first term refers to pressure vessel's final conditions, second to gas in the tank's final conditions) With some *trivial* calculations we finally obtain

$$V_0 = \frac{\gamma p V_p}{p_0 - p_g} \tag{2.3}$$

with

- γ: specific heat ratio of the pressurant
- p: tank pressure [Pa]
- V_p: tank volume [m^3]
- p_0: starting vessel pressure [Pa]
- p_g: final vessel pressure, assumed $1.2p$ [Pa]

Input	**Output**
Ambient temperature [K]	Pressure vessel volume [m^3]
Vessel pressure [Pa]	
Propellant volume [m^3]	
Tank pressure [Pa]	
Pressurant CoolProp name	

Table 2.5: create_pressurant_tank input and output for constant pressure tank.

```
create_propellant_tank(mp0, p0, T0, Q0, propellant, pressurant)
-> return V_tank, V_liq
```

Calculates the tank volume and liquid volume with a given ullage of pressurant (assumes the liquid is kept above vapor pressure). Input and output description in Table **??**.

Input	**Output**
Propellant mass [kg]	Tank volume [m^3]
Tank pressure [Pa]	Liquid volume [m^3]
Ambient temperature [K]	
Vapor quality	
Propellant CoolProp name	
Pressurant CoolProp name	

Table 2.6: create_propellant_tank input and output for constant pressure tank.

```
starting_conditions(mL, T, ptank, ppress, Vtank, Vpress, propellant, pressurant)
->return sL, sG, spress, mG, mpress
```

Calculates the needed input for the step function. Input and output description in Table **??**.

Input at time 0	Output at time 0
Propellant mass [kg]	Liquid specific entropy [J/(kgK)]
Ambient temperature [K]	Ullage gas specific entropy [J/(kgK)]
Tank pressure [Pa]	Pressurant specific entropy [J/(kgK)]
Vessel pressure [Pa]	Ullage mass [kg]
Tank volume [m^3]	Pressurant mass [kg]
Pressure vessel volume [m^3]	
Propellant CoolProp name	
Pressurant CoolProp name	

Table 2.7: starting_condition input and output for constant pressure tank.

2.4 Wrapper

To use one of the three possible tank configurations a series of wrap functions are available in *Tank//tank_ update.py*.

```
build_tank(m, Q, T, oxidizer, pressurant=None, ppress=1e5, p=1e5, plim=None)
-> return masses, volumes, pressures, temperatures, constant_pressure_tank
```

Chooses which function should be used to build the tank. To begin it tries to get the vapor pressure, if it cannot the fluid is super-critic at the given temperature, so it will be full gas (vapor quality overwritten to 1). If it is full gas, it lowers the pressure at vapor pressure if it's higher to avoid liquid formation. If quality is lower than 1 (liquid phase is present) and pressure is higher than vapor pressure, it is a constant pressure tank (flag set as True), else it is self-pressurized or full gas.

```
start_conditions(masses, volumes, pressures, temperatures,
oxidizer, pressurant=None, constant_pressure_tank = False)
-> return ptank, Ttank, mL, entropies_out, masses_out,
pressures_out, temperatures_out
```

Based on the `constant\_pressure\_tank` flag, it selects the needed starting conditions function.

```
update_tank(mdotL, dt, entropies, masses, volumes, pressures, temperatures,
utilities, oxidizer, pressurant=None, constant_pressure_tank = False)
-> return ptank_new, Ttank_new, mL_new, entropies_out, masses_out,
    pressures_out, temperatures_out
```

Performs one time-step based on the `constant\_pressure\_tank` flag.
It should be noticed that, to avoid excessive input and output and hard-to-understand variables, it was chosen to use helper dictionaries for every type of property. They vary with the type of tank and are described in Table **??**

A final consideration is that this kind of wrapping allows for a fouth kind of tank, a pressurised liquid in blow-down from a given pressure. The simple trick is to consider a pressurant line injection area equal to 0. This configuration is meant to allow an easy implementation of the same functions in case of a *bi-liquid propellant engine*.

	Constant pressure	**Full gas or self-pressurizing**
masses	Liquid mass "mL" [kg] Gas mass "mG" [kg] Pressurant mass "mpress" [kg] Vent outflow "mdot_vent" [kg/s]	Total mass "m" [kg] Vapor quality "Q" Liquid mass "mL" [kg] Vapor mass "mG" [kg] Vent outflow "mdot_vent" [kg/s]
entropies	Liquid specific entropy "sL" [J/(kgK)] Gas specific entropy "sG" [J/(kgK)] Pressurant specific entropy "spress" [J/(kgK)]	Liquid specific entropy "sL" [J/(kgK)] Vapor specific entropy "sG" [J/(kgK)] Total entropy "S" [J/K]
pressures	Vessel pressure "ppress" [Pa] Tank pressure "ptank" [Pa] Tank limit pressure "plim" [Pa]	Tank pressure "ptank" [Pa] Tank limit pressure "plim" [Pa]
temperatures	Pressurant temperature "Tpress" [K] Tank temperature "Ttank" [K]	Tank temperature "Ttank" [K]
volumes	Tank volume "Vtank" [m^3] Pressure vessel volume "Vpress" [m^3]	Tank volume "Vtank" [m^3]
utilities	Pressurant line discharge coefficient "CDpress" Pressurant line injection area "Apress" [m^2] Venting valve discharge coefficient "CDvent" Venting valve outlet area "Avent" [m^2]	Venting valve discharge coefficient "CDvent" Venting valve outlet area "Avent" [m^2]

Table 2.8: Helper dictionaries keys and values for different tank configurations.

3. Feeding line

The purpose of the functions is to calculate with higher accuracy the injection pressure using a loss model of the feeding line.

4. Injection

The purpose of the following models is to describe accurately the mass flow injection in different conditions. As for the tank, the type of fluid influences the model to be used: to match the characteristics of a self-pressurising fluid the Non-Homogeneus Non-Equilibrium (NHNE) model [2] was used. It basically condiders a weighted average of the Homogeneus Equilibrium Model, with no slip and multi-phase fluid

$$\dot{m}_{HEM} = A\rho\sqrt{2(h_1 - h_2)} \tag{4.1}$$

and the Single Phase Injection

$$\dot{m}_{SPI} = A\sqrt{2\rho(p_1 - p_2)} \tag{4.2}$$

The weight takes into account the pressure drop and the drop respect to the vapor pressure

$$\chi = \sqrt{\frac{p_1 - p_2}{p_V - p_2}} \tag{4.3}$$

and finally the mass flow can be calculated as

$$\dot{m}_{NHNE} = \frac{\chi}{\chi + 1}\dot{m}_{SPI} + \frac{1}{\chi + 1}\dot{m}_{HEM} \tag{4.4}$$

This model clearly applies to liquid or multi-phase flow, instead with an ideal gas we write

$$\dot{m}_{gas} = \frac{p_1 A}{\sqrt{RT_1}} f(\frac{p_2}{p_1}) \tag{4.5}$$

with

$$f(\frac{p_2}{p_1}) = \sqrt{\frac{2\gamma}{\gamma - 1}((\frac{p_2}{p_1})^{\frac{2}{\gamma}} - (\frac{p_2}{p_1})^{\frac{\gamma+1}{\gamma}})}$$

remembering that a chocked exit section is reached when

$$\frac{p_2}{p_1} \leq (\frac{p_e}{p_1})_{crit} = (\frac{2}{\gamma + 1})^{\frac{\gamma}{\gamma - 1}} \tag{4.6}$$

and in that case we write

$$\dot{m}_{gas} = \frac{p_1 A}{\sqrt{RT_1}}\Gamma \tag{4.7}$$

with

$$\Gamma = \sqrt{\gamma(\frac{2}{\gamma + 1})^{\frac{\gamma+1}{\gamma-1}}}$$

4.1 Functions

The following functions are in *Injection//PyInjection.py*.

```
NHNE_injection(p1, p2, T, cD, fluid)
-> return mdot
```

Calculates mass flux [kg/sm^2] using NHNE model. Returns 0 if $p_1 < p_2$.

```
gas_injection(p1, p2, T, CD, fluid)
-> return mdot
```

Calculates mass flux [kg/sm^2] for an ideal gas using CoolProp properties. Returns 0 if $p_1 < p_2$.

```
gas_injection_custom(p1, p2, T, CD, gamma, M, eps=1.0)
-> return mdot
```

Calculates mass flux [kg/sm^2] for an ideal gas. It is meant to be used for nozzle outflow, using burned gas properties, so it takes in input also the expansion ratio,modifying the chocked condition to

$$f(\frac{p_2}{p_1}) \geq \frac{\Gamma}{\varepsilon} \tag{4.8}$$

Returns 0 if $p_1 < p_2$.

```
massflow(p1, p2, T, CD, fluid)
-> return mdot
```

Chooses the model to use with a *try/except* structure, because the calculation of vapor pressure using CoolProp will fail if it is super-critical.

All the previous functions take into account the effects of the discharge coefficient.

5. Performance

The purpose of this function is to calculate the performances for a certain condition and handle the needed calculations for the following functions. The calculation is based on the following assumptions:

- the combustion is in chemical equilibrium
- infinite area combustor
- frozen equilibrium at throat section

To perform this calculations NASA CEA [4] and its Python wrapper RocketCEA [6] were used.

5.1 Functions

The following functions are available in Performance//performance_singlepoint.py.

```
calculate_performance(Ainj, Aport, Ab, eps, ptank, Ttank, pc, CD,
                          a, n, rho_fuel, oxidizer, fuel, pamb=0.0, gamma0=1.3)
-> return (p_inj, mdot_ox, mdot_fuel, mdot, Gox, r, MR, Tc, MW, gamma, eps_out, cs,
           CF_vac, CF, Ivac, Is, flag_performance)
```

Calculates the line losses and the injected mass flow, uses it to get oxidizer mass flux for the regression rate

$$r = aG_{ox}^{n} \tag{5.1}$$

to calculate the fuel mass flow

$$\dot{m}_{fuel} = \rho_f A_b r \tag{5.2}$$

from it the mixture ratio can be calculated as

$$MR = \frac{\dot{m}_{ox}}{\dot{m}_f} \tag{5.3}$$

With that we have the three needed inputs for the CEA calculations (chamber pressure, mixture ratio and expansion ratio).
It is possible to ask for the adapted expansion ratio passing `eps="adapt"`, and it will be calculated as

$$\varepsilon = \frac{\Gamma}{f(\frac{p_{amb}}{p_c})} \tag{5.4}$$

with

$$\Gamma = \sqrt{\gamma(\frac{2}{\gamma+1})^{\frac{\gamma+1}{\gamma-1}}}$$

$$f(\frac{p_{amb}}{p_1}) = \sqrt{\frac{2\gamma}{\gamma-1}((\frac{p_{amb}}{p_1})^{\frac{2}{\gamma}} - (\frac{p_{amb}}{p_1})^{\frac{\gamma+1}{\gamma}})}$$

```
pressure_fun(Ainj, Aport, At, Ab, eps, ptank, Ttank, pc, CD, a, n, rho_fuel,
   oxidizer, fuel, pamb=0.0, gamma0=1.3)
-> return Fpc
```

From the definition

$$c^* = \frac{p_c A_t}{\dot{m}} \tag{5.5}$$

and we use it to calculate the pressure function to find the steady state pressure for the optimisation:

$$F(p_c) = \frac{\dot{m} c^*}{A_t} - pc \tag{5.6}$$

6. Optimisation

The purpose of these functions is to find the best configuration for a certain fuel/oxidizer combination. The optimisation method used is a global optimisation, in a range of the following geometrical parameters:

$$\frac{D_{inj}}{D_t} \in (0, 1) \tag{6.1}$$

$$\frac{D_p}{D_t} > 1 \tag{6.2}$$

$$\frac{L_c}{D_t} > 1 \tag{6.3}$$

which are the injection (equivalent) diameter, port diameter and grain length.
For each configuration it is needed to find the stationary pressure, the one reached when inlet and outlet mass flow are equal.

6.1 Functions

The following functions are available in *Optimization//optimization.py*.

```
starting_pressure(Ainj, Aport, At, Ab, eps, ptank, Ttank, CD, a, n,
    rho_fuel, oxidizer, fuel, pamb=0.0, gamma0=1.3)
-> return pc_best
```

Finds the best pressure to start the calculation to find the chamber pressure. It is found in a range of 200 equally spaced points between p_{amb} and $0.8p_{inj}$ and other 200 points between $0.8p_{inj}$ and p_{inj}. This particular division is due to the fact that typically the zero of the pressure function **??** is in the final part. To find the best start, for each pressure the pressure function is calculated and the one with the minimum absolute value is selected.

```
get_pressure(Ainj, Aport, At, Ab, eps, ptank, Ttank, CD, a, n,
   rho_fuel, oxidizer, fuel, pamb=0.0, gamma0=1.3)
-> return pc, Fpc, n_iter, maxit, gamma0
```

Finds the zero of the pressure function **??** using a Newton-Raphson method with a fallback measure that increments the slope of the tangential if the point falls out of the range (p_{amb}, p_{inj}).

```
full_range_simulation(Dport_Dt_range, Dinj_Dt_range, Lc_Dt_range, eps, ptank, Ttank,
     CD, a, n, rho_fuel, oxidizer, fuel, pamb=0.0, gamma0=1.3)
-> return (pc_array, Fpc_array, p_inj_array, mdot_ox_array, mdot_fuel_array,
   mdot_array, Gox_array, r_array, MR_array, eps_array, Tc_array,
   MW_array, gamma_array, cs_array, CF_vac_array, CF_array, Ivac_array,
   Is_array, flag_array)
```

Finds the steady pressure and calculates all the perfomances for each configuration, given by the combination of the three parameters. Returns Numpy arrays of size $(n_{D_p}, n_{D_{inj}}, n_{L_c})$ with all the performances and needed values.

6.2 Dimensionalization functions

The following functions are meant to get the real configuration given a mission or geometrical requirement. Available in *Optimisation//dimensionalize_parameters.py*.

```
Thrust2Rocket(F, Dinj_Dt, Dp_Dt, Lc_Dt, mdot_ox_Dtsq, mdot_fuel_Dtsq, mdot_Dtsq, pc, CF,
-> return Dinj, Dp, Lc, Dt, mdot_ox, mdot_fuel, mdot
```

Calculates the throat area as

$$At = \frac{F}{\eta_{CF} C_F} \frac{1}{p_c} \tag{6.4}$$

and then uses the diameter for the dimesnsionalization. Returns also the mass flows for completeness.

```
Dt2Rocket(Dt, Dinj_Dt, Dp_Dt, Lc_Dt, mdot_ox_Dtsq, mdot_fuel_Dtsq, mdot_Dtsq)
-> return Dinj, Dp, Lc, mdot_ox, mdot_fuel, mdot

Dinj2Rocket(Dinj, Dinj_Dt, Dp_Dt, Lc_Dt, mdot_ox_Dtsq, mdot_fuel_Dtsq, mdot_Dtsq)
-> return Dt, Dp, Lc, mdot_ox, mdot_fuel, mdot

Dp2Rocket(Dp, Dinj_Dt, Dp_Dt, Lc_Dt, mdot_ox_Dtsq, mdot_fuel_Dtsq, mdot_Dtsq)
-> return Dt, Dinj, Lc, mdot_ox, mdot_fuel, mdot
```

These functions calculate the geometries and mass flows given a geometrical requirement.

7. Grain geometry

The purpose of these functions is to give the possibility to manage complex and helical geometries, their evolution through combustion and to perform needed calculations.

7.1 Calculation functions

To perform the needed operations the following functions were built, available in *Geometry//geometry_ calculation.py*.

```
create_regular_poligon(n_sides, circum_radius)
-> return x_poly, y_poly
```

Creates a centered, regular polygon inscribed in the circumference. Returns Numpy [3] arrays of size $(n,)$ composed of x and y values of the vertices.

```
create_repeated_instance(x, y, n_instances)
-> return x_u, y_u
```

Given an instance of points as two Numpy arrays of size $(n,)$, repeats it *n_ instances* times. Points sharing the same position are filtered returning only uniques. Returns Numpy arrays of size $(n,)$ composed of x and y values of the figure.

```
translate_figure(x, y)
-> return x_translated, y_translated
```

Positions the origin in the figure's centroid. Returns Numpy arrays of size $(n,)$ composed of x and y values of the translated figure.

```
sort_input(x, y, z=1)
-> return x_sorted, y_sorted
```

Sorts the points in counter-clockwise order if `z=1` or clockwise if `z=-1`, with z rotation axis' versor. Returns Numpy arrays of size $(n,)$ composed of sorted x and y values of the figure.

```
fill_borders(x, y, n_points_per_side)
-> return x_fill, y_fill
```

Fills the segment between each consecutive point with input number of points. Returns Numpy arrays of size $(n,)$ composed of x and y values of the figure.

```
fill_borders_circumference(x, y, n_points_per_side)
-> return x_fill, y_fill
```

Fills between every consecutive point at the same radius with an arc of angle-wise linearly spaced points with the same radius. In other cases fills the segments. Returns Numpy arrays of size $(n,)$ composed of x and y values of the figure.

```
calculate_surfaces_from_points(x, y, lc, step=0.0)
-> return PortArea, BurningArea
```

Calculates the base and side area of a parallelepipedon with base given by `x,y` points and height equal to `lc`. It is possible to consider a twisted body giving a non-zero value to `step`, which is the axial distance during a 360 turn. The function closes the polygon (it is not needed to repeat the first point), calculates the module of every side (linear approximation of circumferences: it should be used with filled geometries) and the radius of each vertex.
The port (base) area is calculated as the sum of each triangle's area (radius-side-radius) using Heron of Alexandria's formula:

$$A = \sqrt{p \cdot (p-a) \cdot (p-b) \cdot (p-c)} \tag{7.1}$$

with:

- p: half-perimeter of the triangle
- a, b, c: sides of the triangle

The side area is calculated as

$$A = 2p \cdot l_c \tag{7.2}$$

for not twisted bodies. Instead with twisted bodies we divide the height in small units

$$h = min(min(c), step)/10 \tag{7.3}$$

with c sides of the figure, rotate each point of the rotation at that height

$$d\theta = 2\pi \frac{h}{step} \tag{7.4}$$

Then we get the distance between each translated point and the starting side with Pythagoras' theorem using height h and the distance between the projection of the translated point (T) on the base plane and a straight line formed by the original point (A) and its following (B):

$$H = \sqrt{h^2 + s^2} \tag{7.5}$$

with

$$s = \frac{|(x_B - x_A)(y_T - y_A) - (y_B - y_A)(x_T - x_A)|}{\sqrt{(y_B - y_A)^2 + (x_B - x_A)^2}}$$

Finally, we find the area of each parallelogram, sum them and multiply by the number of height sections

$$A = \frac{l_c}{h} \sum H \cdot c \tag{7.6}$$

```
fill_and_calculate_surfaces_and_volume(x, y, lc, n_pointsperside=20,
circular=False, step=0.0, Vol_prechamber=0.0, Vol_postchamber=0.0)
-> return A_port, A_burn, Vol_chamber
```

This function wraps the previous, filling the borders using the self-evident flag `circular` to know which function choose, calculates port area and burning area and calculates the internal fluid volume of the chamber as

$$V = V_{pre} + A_p l_c + V_{post} \tag{7.7}$$

allowing to consider the effects of pre-chamber and mixer.
The main reason of this function is to have an easy call for the needed calculation considering that to update the geometry the linear and circular fills are removed to burn the actual sides without errors, as it will be later discussed.

```
calculate_fuel_mass(Ap, lc, D_chamber, fuel_density)
-> return mfuel
```

Calculates fuel mass as

$$m_{fuel} = \rho_{fuel}(\frac{\pi}{4} D^2_{chamber} - A_p) l_c \tag{7.8}$$

7.2 Update functions

To update the geometry through the combustion process the following functions were built, available in *Geometry//geometry*$_u$*pdate.py*.

```
remove_close_vertices(x, y, tol=1e-12)
-> return pts2[:, 0], pts2[:, 1]
```

Removes points closer than the tolerance `tol` respect to the ones before. Returns Numpy arrays of size $(n,)$ composed of non-close x and y values of the figure.

```
remove_collinear_simple(x, y, tol=1e-12)
-> return x_new, y_new
```

Removes a point if it is collinear with the one before and the one after. Returns Numpy arrays of size $(n,)$ composed of x and y vertex of the figure.

```
remove_samearc_simple(x, y, tol=1e-12)
-> return x_new, y_new
```

Removes a point if it is at the same radius of the one before and the one after. Returns Numpy arrays of size $(n,)$ composed of non-circumferential x and y of the figure.

```
_line_intersection_from_dirs(pA, vA, pB, vB, eps=1e-12)
-> return bool, pt, s, t, cond
```

Solves a linear system to find the intersection between two lines starting from point A and B knowing the directions. Returns a flag to know if the solution was found, the intersection point, the distances from the two points and the condition number. Considering that the two directions are towards the expected intersection, distances `s` and `t` should be positive.

```
burn_surface(x, y, z, regression_rate, dt, min_param=1e-9,
parallel_dot_thresh=0.9999, close_tol=1e-12, merge_tol=1e-9)
-> return x_new, y_new
```

Removes collinear points to isolate the vertex, finds the direction between each point and it's following. `z` represents the rotation axis for the order of the points (1 if couter-clockwise and -1 if clock-wise) and is needed to find the normal direction for burning, in fact for counter-clockwise order we have

$$t = (t_x, t_y) \tag{7.9}$$

$$n = (t_y, -t_x) \tag{7.10}$$

and for clock-wise, instead

$$n = (-t_y, t_x) \tag{7.11}$$

So we can write

$$n = (zt_y, -zt_x) \tag{7.12}$$

We finally find the middle points of each segment and translate it of a vector

$$r_n = rdt(zt_y, -zt_x) \tag{7.13}$$

To find the intersections it is necessary to discern between sides with obtuse and acute angles, calculated using the cross product of the two normals

$$z(n_A \times n_B) = \begin{matrix} > 1 & obtuse\ angle \\ < 1 & acute\ angle \end{matrix} \tag{7.14}$$

For obtuse angles it is enough to do the intersection, instead with acute angles we have a cusp and need to know if it does not degenerate. It happens when the cusp becomes a single side and could degenerate changing the position of two following translated mid-points. This condition is verified when the side of one of the two sides is lower than the critical distance

$$L_{crit} = \frac{rdt}{tan\frac{\theta}{2}} \tag{7.15}$$

with θ angle between the two sides. If this condition is verified, to perform the intersection we remove the too small sides and do it with the external one(s).
As a fallback condition in case the intersection is not found, we use the midpoint between the two points, but only for a cusp, in the other case we move the actual vertex in the two directions.
The minimum amount of non-collinear points to have a non-empty solution is 3 (triangle)
Returns Numpy arrays of size $(n,)$ composed of the moved x and y of the figure.

```
burn_surface_circular(x, y, z, regression_rate, dt, min_param=1e-9,
parallel_dot_thresh=0.9999, close_tol=1e-12, merge_tol=1e-9)
-> return x_new, y_new
```

This function works equally as the previous one but filters also the points at the same radius. There is a mask for the sides that are circumference arcs, but the intersection method does not change, moving the midpoints on the chord of the arcs. The fallback condition changes for the points at the sides of an arc, moving them only radially.
Moreover this function accepts in input Numpy arrays of size $(1,)$ which represent a circumference.
Returns Numpy arrays of size $(n,)$ composed of the moved x and y of the figure.

```
burn_grain(x, y, z, regression_rate, dt, circular=False, min_param=1e-9,
parallel_dot_thresh=0.9999, close_tol=1e-12, merge_tol=1e-9)
-> return x_out, y_out
```

Wrapper function for easy calls to decide which burn function to use based on the `circular` flag.
Both functions have a series of helpers to remove self-intersecting or too-close points and avoid errors in the geometry.
The performances were proven solid with high regressions and hundreads of iterations.

7.3 Dimensionalization functions

The following functions are meant to pass from an input geometry to a real one using a certain requirement. In particular they are meant to be used after the optimization, already discussed. They are available in *Geometry//dimensionalize.py*.

```
dimensionalize_geometry(x, y, Dp, Lc, p=0.0)
-> return x_1, y_1, L_1, p_1
```

Matches the required port area and burning area while keeping $\frac{p}{D_{eq}}$ ratio. To do so we calculate the port area of the initial geometry using `calculate_surfaces_from_points(...)` and find the equivalent diameter

$$D_{eq,0} = \sqrt{\frac{4A_p}{\pi}} \tag{7.16}$$

and use it to find the needed values

$$\begin{aligned} x_1 &= \frac{D_p}{D_{eq,0}} x_0 \\ y_1 &= \frac{D_p}{D_{eq,0}} y_0 \\ p_1 &= \frac{D_p}{D_{eq,0}} p_0 \end{aligned} \tag{7.17}$$

Finally, we use a reference length L_{mid} for the geometry ($= p_1$ if non-zero or $= 0.5L_c$), calculate the burning area using `calculate_surfaces_from_points(...)` and find the actual length

$$L_{c,1} = L_{mid} \frac{\pi D_p L_c}{A_b(x_1, y_1, L_{mid}, p_1)} \tag{7.18}$$

Returns the new geometry points x and y values as Numpy arrays of size $(n,)$, grain length and step.

8. Mission

The purpose of these functions is to simulate a mission with certain accuracy and to configure an engine based on the required mission.

8.1 Update chamber functions

The following functions were built to have an easy call in the main mission function to update pressure and thermodynamic properties at every step. Available in *Mission//chamber_ update.py*.

```
find_dt(Tc, MW, gamma, Tc_CEA, MW_CEA, gamma_CEA,
   m_c_i, Dmdot_in, Dmdot_out, tol=1e-3)
-> return dt
```

Called inside the next function, returns the needed time step to have a variation in the thermodynamic properties under the required tolerance. It is calculated using the mixture of gases formulas:

$$\begin{aligned} m_{i+1} &= m_i - \dot{m}_{out}dt + \dot{m}_{in}dt \\ c_{p,i+1} &= \frac{(m_i - \dot{m}_{out}dt)c_{p,i} + \dot{m}_{in}dtc_{p,CEA}}{m_{i+1}} \\ T_{c,i+1} &= \frac{(m_i - \dot{m}_{out}dt)T_{c,i}c_{p,i} + \dot{m}_{in}dtT_{c,CEA}c_{p,CEA}}{m_{i+1}} \\ MW_{i+1} &= \frac{m_{c,i+1}}{\frac{m_i - \dot{m}_{out}dt}{MW_i} + \frac{\dot{m}_{in}dt}{MW_{CEA}}} \end{aligned} \tag{8.1}$$

Setting a relative tolerance for the generic value

$$|\frac{X_i + 1}{X_i} - 1| = tol \tag{8.2}$$

we can finally find four different time-steps, leading us to the decision of taking the minimum one.

```
update_Temperature_and_gasproperties(pc, Tc, MW, gamma, Tc_CEA, MW_CEA,
          gamma_CEA, mdot_ox, mdot_fuel, mdot_throat,
          Vol_chamber, tol=1e-3)
-> return Tc_actual, MW_actual, gamma_actual, m_c, dt
```

Calculates new gas properties using ideal gas state equation and mixture of gases relations in ??. The first step is to calculate the mass of fluid inside the chamber

$$m_i = \frac{p_{c,i}V_i}{R_iT_{c,i}} \tag{8.3}$$

we get the time-step with `find_dt(...)` and then apply the ??.
What should be considered while reading the results is that while the ideal gas assumption could be acceptable, the results to update the values are taken from NASA CEA [4], which

considers the reaction in chemical equilibrium with infinite residence time and does not consider transitory effects due to finite distances and times. This makes this model less-accurate than a proper CFD with reactive species but allows to use a vast database of possible reactants. The main solution is to validate the output through static-fires, determining performances' efficiencies and using them for better prediction.

```
update_chamberpressure(m_c, Tc, MW, Vol_chamber)
-> return pc
```

Calculates the new chamber pressure using ideal gas state equation with updated values.

8.2 Simulation functions

The following functions are intended to fully simulate a mission using required times and injection or burning conditions. Available in *Mission//mission_ simulation.py*.

```
find_nozzle_output(gamma, MW, Tc, pc, mdot_throat, pamb, At, eps)
return Me, Te, pe, Ve
```

calculates the properties at the exit section of the nozzle, needed to calculate the thrust. We determine if the nozzle is chocked when

$$\frac{p_{amb}}{p_c} < (\frac{p_e}{p_c})_{crit} = (\frac{2}{\gamma+1})^{\frac{\gamma}{\gamma-1}} \tag{8.4}$$

but actually this relation is valid if we look for a chocked exit section. The real condition is a critical throat section, from mass conservation we get

$$f(\frac{p_{amb}}{p_c}) \geq \frac{f(M=1)}{\varepsilon} \tag{8.5}$$

If one of the two conditions is satisfied, we iteratively find exit section Mach number using mass flow conservation and mach function equation to create the target function

$$F(M_e) = f(M_e) - \dot{m}_t \frac{\sqrt{RT_c}}{p_c A_t \varepsilon} = 0 \tag{8.6}$$

with

$$f(M) = \frac{\sqrt{\gamma} M}{\sqrt{(1+\frac{\gamma-1}{2}M^2)^{\frac{\gamma+1}{\gamma-1}}}}$$

We use Newton's method, calculating the derivative of Mach function

$$\frac{df}{dM} = \sqrt{\gamma}(\frac{1}{\sqrt{(1+\frac{\gamma-1}{2}M^2)^{\frac{\gamma+1}{\gamma-1}}}} - \frac{\gamma+1}{2}M^2 \frac{(1+\frac{\gamma-1}{2}M^2)^{\frac{2}{\gamma-1}}}{\sqrt{(1+\frac{\gamma-1}{2}M^2)^{3\frac{\gamma+1}{\gamma-1}}}}) \tag{8.7}$$

After finding the exit Mach number we calculate the needed values using the adiabatic flow equations

$$\begin{aligned} \tfrac{T_c}{T_e} &= 1+\tfrac{\gamma-1}{2}M^2 \\ \tfrac{p_c}{p_e} &= (1+\tfrac{\gamma-1}{2}M^2)^{\frac{\gamma}{\gamma-1}} \\ v &= M\sqrt{\gamma R T_c} \end{aligned} \tag{8.8}$$

If the nozzle is not chocked we assume

$$p_e = p_{amb} \tag{8.9}$$

and use the ??.

```
get_starting_conditions(pamb, Tamb, rho_fuel, x, y, Lc, D_chamber,
       Vol_prechamber, Vol_postchamber, masses, volumes,
       pressures, temperatures,
       oxidizer, pressurant=None, pitch=0.0, circular=False,
       npointsperside=50, constant_pressure_tank=False)
-> return (pc, mdot_throat, Tc, MW, gamma,
            ptank, Ttank, mL, m_fuel,
            entropies, masses, pressures, temperatures,
            Ap, Ab, Vol_chamber)
```

Sets starting pressure, temperature and properties equal to ambient's (CoolProp's Air), gets tank starting conditions with `start_conditions(...)`, calculates port area, burning area and chamber volume with `fill_and_calculate_surfaces_and_volume(...)` and the fuel mass with `calculate_fuel_mass(...)`.

```
run_one_step_no_burn(pc, mdot_throat, Tc, MW, gamma, rho_fuel, pamb, ptank,
      Ttank, eps, Ainj, At, Ap, Ab, Lc, Vol_chamber, D_chamber,
      x, y,
      entropies, masses, volumes, pressures, temperatures, utilities,
      CD, oxidizer,
      pressurant=None, rend_CF = 1.0,
      constant_pressure_tank=False, tol=1e-3)
-> return (pc, mdot_throat, Thrust, dt,
            Tc, MW, gamma, ptank, Ttank, eps,
            mL, m_fuel,
            Ap, Ab, Vol_chamber,
            x, y,
            entropies, masses, pressures, temperatures,
            performances)

run_one_step(pc, mdot_throat, Tc, MW, gamma, a, n, rho_fuel, pamb, ptank,
    Ttank, eps, Ainj, At, Ap, Ab, Lc, Vol_chamber, Vol_prechamber,
    Vol_postchamber, D_chamber, x, y, z, entropies, masses, volumes,
    pressures, temperatures, utilities,
    CD, oxidizer, fuel, pressurant=None,
    rend_cstar = 1.0, rend_CF = 1.0, pitch=0.0,
    circular=False, npointsperside=50,
    constant_pressure_tank=False, tol=1e-3)
-> return (pc, mdot_throat, Thrust, dt,
            Tc, MW, gamma, ptank, Ttank, eps,
            mL, m_fuel,
            Ap, Ab, Vol_chamber,
```

```
x, y,
entropies, masses, pressures, temperatures,
performances, flag)
```

These two functions have the same output and are meant to be used inside an iteration (to have an easy call) and manage all the operations to update the values after one time-step. The explicit outputs are used directly in the next iteration or to control the termination, the implicit one `performances` represents a dictionary with a series of important values to be given as an output. The workflow for the mission simulation is the one presented in Figure **??**. The main difference between the burn and no-burn function are in point *2)*, where only `linelosses(...)` and `massflow(...)` functions are used and the fluid properties are the oxidizer ones, and point *4)*, that is skipped. The `performances` dictionary is built as in Table **??**.

Common values	
Chamber pressure "pc" [Pa]	Burning area "Ab" [m^2]
Injection pressure "p_inj" [Pa]	Chamber internal volume "Vol_chamber" [m^3]
Time step "dt" [s]	Fuel mass "m_fuel" [kg]
Thrust "Thrust" [N]	Liquid mass in the tank "mL" [kg]
Oxidizer massflow "mdot_ox" [kg/s]	Tank pressure "ptank" [Pa]
Fuel massflow "mdot_fuel" [kg/s]	Tank temperature "Ttank" [K]
Total inlet massflow "mdot" [kg/s]	Exit Mach "Me"
Total outlet massflow "mdot_throat" [kg/s]	Exit temperature "Te" [K]
Chamber total temperature "Tc" [K]	Exit pressure "pe" [Pa]
Molecular weight "MW" [kg/kmol]	Ambient pressure "pamb" [Pa]
Specific heat ratio "gamma"	Entropy dictionary "entropies"
Expansion ratio "eps"	Mass dictionary "masses"
Grain points x-coordinates "x" [m]	Pressure dictionary "pressures"
Grain points y-coordinates "y" [m]	Temperature dictionary "temperatures"
Port area "Ap" [m^2]	

No burn	**Burn**
	Oxidizer mass flux "Gox" [kg/(m^2s)]
	Regression rate "r" [m/s]
	Mixture ratio "MR"
	Chamber temperature from CEA "Tc_CEA" [K]
	Molecular weight from CEA "MW_CEA" [kg/kmol]
	Specific heat ratio from CEA "gamma_CEA"
	Characteristic velocity (c*) "cstar" [m/s]
	Thrust coefficient in vacuum "CFvac"
	Thrust coefficient "CF"
	Specific impulse in vacuum "Ivac" [s]
	Specific impulse "Is" [s]
	CEA output flag "flag"
Burn flag "burn": False	Burn flag "burn": True

Table 8.1: Performance keys and values for burn and no-burn conditions.

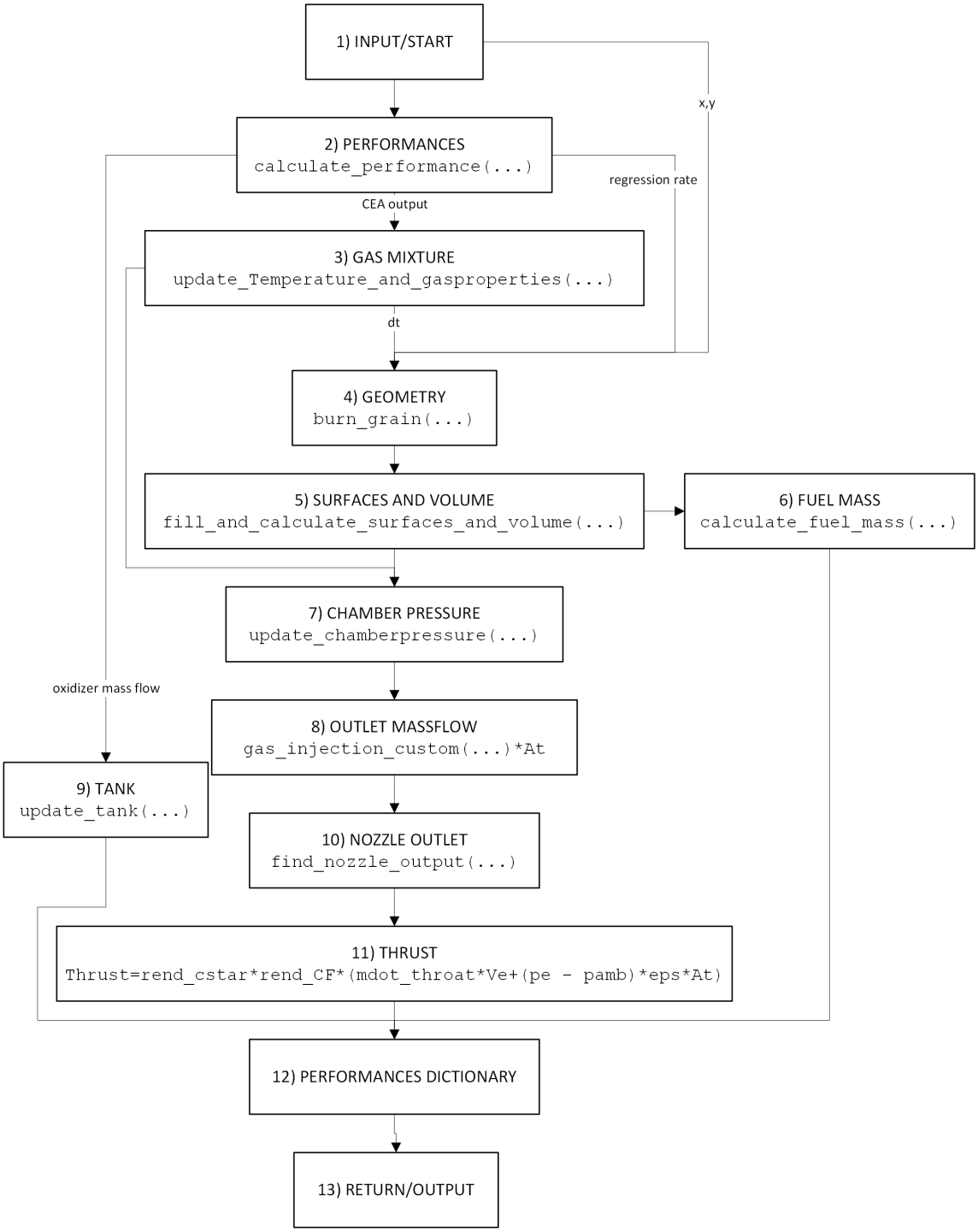

Figure 8.1: Mission step flowchart

```
run_full_mission(burn_time, pamb, Tamb, a, n, rho_fuel,
                     eps, Ainj, At, Lc, D_chamber,
                     x, y, z,
                     Vol_prechamber, Vol_postchamber,
                     masses, volumes, pressures, temperatures, utilities,
                     CD,
                     oxidizer, fuel, pressurant=None,
                     rend_cstar = 1.1, rend_CF = 1.1,
                     pitch=0.0, circular=False, delay_time = 0.0, npointsperside=50,
      constant_pressure_tank=False,
                     tol=1e-3)
->return time, performances_out, out_log
```

Runs the step functions while the conditions are satisfied. First phase is delaied ignition (no burn), then combustion and at the end a final emptying of the tank. The conditions to be satisfied are the following:

- $mL > 0$ or `full_gas_tank`
- $m_{fuel} > 0$
- $max(\sqrt{x^2 + y^2}) < 0.5 D_{chamber}$

These are later used to check why the iteration stopped and write `out_log`.

```
run_full_mission_iteration(burn_time, pamb, Tamb, a, n, rho_fuel,
                     eps, Ainj, At, Lc, D_chamber,
                     x, y, z,
                     Vol_prechamber, Vol_postchamber,
                     masses, volumes, pressures, temperatures, utilities,
                     CD,
                     oxidizer, fuel, pressurant=None,
                     rend_cstar = 1.1, rend_CF = 1.1,
                     pitch=0.0, circular=False, delay_time = 0.0, npointsperside=50, con
                     tol=1e-3)
-> return time, performances_out, out_log
```

Same as the previous function but there is no final emptying and `out_log` is composed of bools for convergence verification to match the mission.

```
match_mission(burn_time, pamb, Tamb, a, n, rho_fuel,
                     eps, Ainj, At, Lc, D_chamber,
                     x, y, z,
                     Vol_prechamber, Vol_postchamber, utilities,
                     CD,
                     mtank, Q,
                     oxidizer, fuel, pressurant=None,
                     rend_cstar = 1.1, rend_CF = 1.1,
                     pitch=0.0, circular=False, delay_time = 0.0, npointsperside=50,
                     tol=1e-3, ppress=1e5, ptank0=1e5, plim=None)
-> return time, inputs, performances_out, out_log
```

Runs `run_full_mission_iteration(...)` while the required times are not reached with mass exceedings higher than 5%. To increase a certain value it is used a time proportional coefficient to update the starting value

$$X_{new} = X_{old} \frac{t_{target}}{t_{real}} \tag{8.10}$$

To simplify the description, a flowchart of the logic is shown in Figure **??**. It is important to notice that this function builds the tank, while the mission functions ask for a built tank, allowing the user to simulate a mission with a generic configuration.

```
normalize_performances(list_of_dict)
-> return dict_of_list
```

The output of the mission simulation is a list of performances dictionaries, this helper function transforms it in a dictionary with the same keys and a list of values in the output. If the key does not appear the value is *None*, to avoid mismatches with burn and no-burn differences.

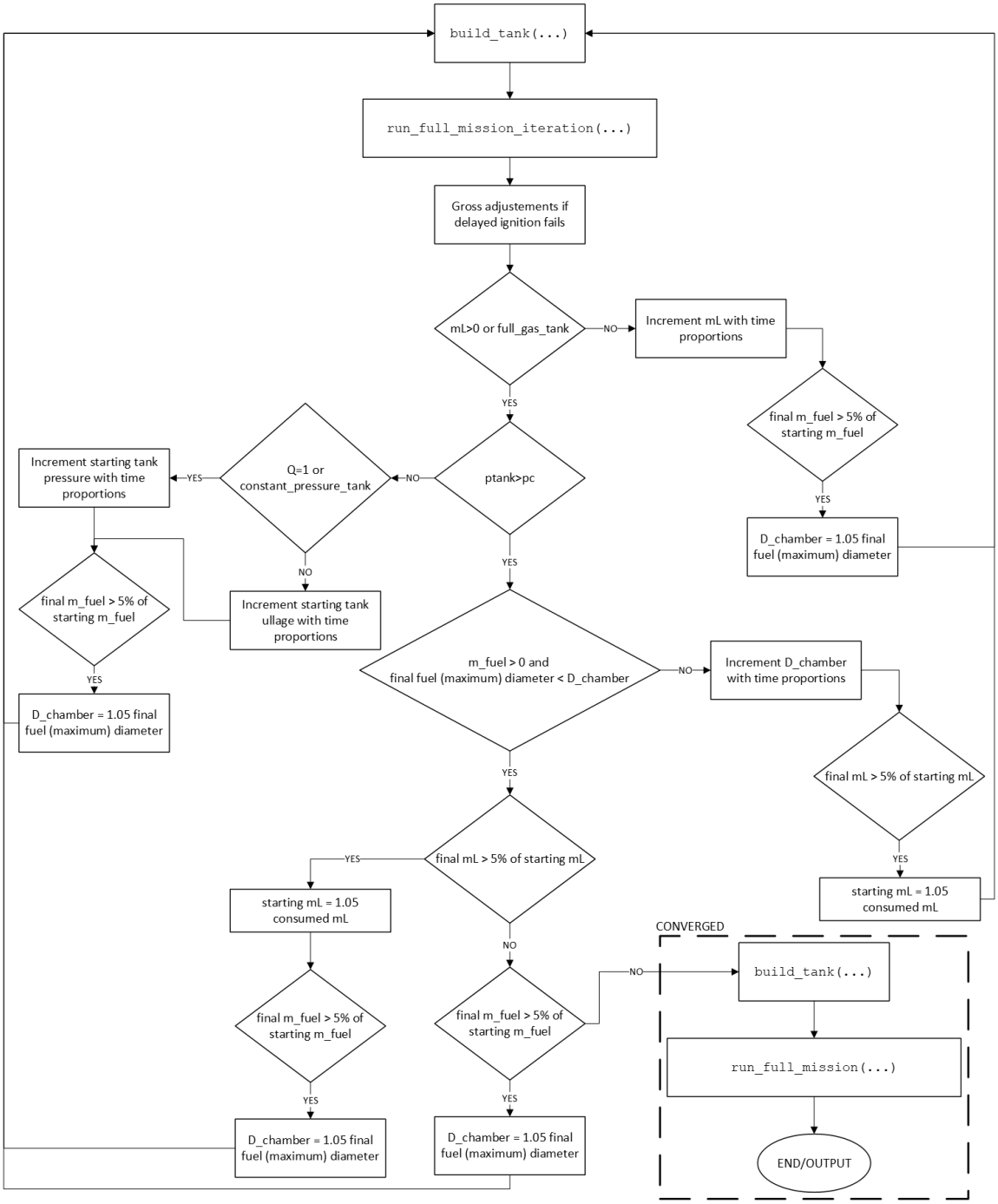

Figure 8.2: Mission match flowchart

9. Conclusions

The presented work is what will lead to the design of an hynrid rocket engine, but has a lot of limits and needs futher developements. The main assumptions are:

- Chemical equilibrium for combustion
- 0-D combustion chamber
- Burned gas acts as ideal

Future work:

- Surpass chemical equilibrium for combustion
- Grain matching for a given tank
- Multi-port grain
- 1-D combustion with losses calculation
- Include injector dimensioning using 1-D models
- Include nozzle dimensioning with CEA model selecting shifting or frozen equilibrium

9. Bibliography

[1] Ian H. Bell et al. "Pure and Pseudo-pure Fluid Thermophysical Property Evaluation and the Open-Source Thermophysical Property Library CoolProp". In: *Industrial & Engineering Chemistry Research* 53.6 (2014), pp. 2498–2508. DOI: 10.1021/ie4033999. eprint: http://pubs.acs.org/doi/pdf/10.1021/ie4033999. URL: http://pubs.acs.org/doi/abs/10.1021/ie4033999.

[2] Brian J. Cantwell Benjamin S. Waxman Jonah E. Zimmerman and Gregory G. Zilliac. "Mass Flow Rate and Isolation Characteristics of Injectors for Use with Self-Pressurizing Oxidizers in Hybrid Rockets". In: *AIAA/ASME/SAE/ASEE Joint Propulsion Conference (JPC 2013)* AIAA 2006-4504.3 (July 14, 2013), p. 16. DOI: 20190001326.

[3] Charles R. Harris et al. "Array programming with NumPy". In: *Nature* 585.7825 (Sept. 2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2. URL: https://doi.org/10.1038/s41586-020-2649-2.

[4] Bonnie J. McBride Sanford Gordon. "Computer program for calculation of complex chemical equilibrium compositions and applications. Part 1: Analysis". In: *NASA Technical Report Services* (1994).

[5] Spencer N. Chandler Stephen A. Whitmore. "Engineering Model for Self-Pressurizing Saturated-N2O-Propellant Feed Systems". In: (). DOI: 10.2514/1.47131.

[6] Charlie Taylor. "RocketCEA Python library". In: ().